Contents

Contents 1
1. Introduction 1
2. Generative AI 2
3. Submission Requirements 2
Code 2
Written Paper 3
Individual submission 4
4. Marking Criteria 5
Complexity of the implemented agent 5
General clarity 6
Tournament 6
Experiments 6

1. Introduction

The first assignment for ECS7032P consists of developing an AI agent for the game Sushi Go! in TAG. You will work in groups of 3 students. You can use any of the search techniques that have been seen so far during the module, i.e. tree or evolutionary search.

You can find an **introductory video** about Sushi Go! and the **submission links** below. Submission is done in groups, and you can find the **group selection activity** on the front page of the QM+ module page.

The goal of the assignment is to **create or modify one agent**, which you must compare against other agents in the framework (or previous iterations of the same agent you submit or modify), **describing your findings in a four-page report**. For instance, you can iteratively build improvements on a given agent and report how these improvements made the agent better. All agents will be put together in a **competition** that will grant extra marks to the best agents.

Before attempting this assignment, you are strongly encouraged to do all the Exercises seen in the labs about TAG, so you can familiarise yourself with the framework and the concepts detailed in this description.

2. Generative AI

IMPORTANT Note on the use of generative systems (Claude, ChatGPT, Co-pilot, etc). The use of any of this tools that generate text or code is **NOT** forbidden. We do establish the following rules for this assignment:

- If you do use any of these, you must acknowledge them. You should put this section at the end of your report, either before or after your references. These last two sections (generative AI use and references) do not count against the four-page limit – the total report must still be within five pages.
- Failing to acknowledge the use of AI-generated text will mean losing marks in the assignment. Fraudulent reporting of source material qualifies as academic misconduct (see the latest academic misconduct policy): for example, if you provided an inaccurate summary of an existing paper written by a generative model, that would be considered fraudulent reporting of source material, which in turn qualifies as academic misconduct.
- Any sections of the report that use AI-generated text shall have a citation to the AI system used to generate the text and include the prompt(s) used to generate that text.
- You are 100% responsible for what is written in your report and your code. If there are any mistakes derived from the use of these tools, these will be your mistakes and will affect your marks anyway.

3. Submission Requirements

Remember that you must be part of a group in QMplus before submitting your assignment. Submission of this assignment includes **a group** and **an individual** component.

The group submission consists of two parts: the **code** of your agent and a **report** with your findings.

Code

Code of the agent. The code will be assessed upon its correctness and clarity, as well as the presence of comments that explain the most critical parts of the agent. Note the following:

1. You are **not** allowed to use, copy or modify the advanced MCTS agent (`players.mcts.MCTSPlayer`) for implementing your agent submission. You are allowed to use any of `players.mcts.BasicMCTS`, `players.rmhc.RMHCPlayer`, `players.rhea.RHEAPlayer`, `players.search.MaxNSearch` or anything in the `players.simple` package as a basis for development. If in doubt, ask one of the teaching team.

   To set up a new agent we recommend you create a new package and copy the code from one of the base agents into this package. You can then change the code in this package to adjust the agent for Sushi Go. You will need to update the references within the package if you do this, for example you will need to ensure that your new copy of BasicMCTSPlayer refers to your new copy of BasicTreeNode in its imports, and does not still rely on the original (unless you want it to).

2. Implementing an entirely new MCTS from scratch is also not advisable. We will interpret this to mean that you just asked an LLM to do it. Sensible modifications of the existing algorithms above will do better.

3. The code must include a **README File** that describes how to run the agent (i.e. how to call the constructor of the agent, plus any other initialisation and parameter instructions) and a **JSON configuration file** for the agent to use for the competition.
4. All that is needed to run your agent must be in a package with the name "**group{GROUP_LETTER}**" (e.g. package "groupA"). No modifications that you make outside of this package will be carried over for marking and running the competition (this is, you can't change the game, or the TAG framework that runs it). You are strongly encouraged to double check that you can run your agent in a fresh version of the repository on its own, before submission.

A competition will be run between all submitted agents. This competition will be run in the 3-player version of Sushi Go! The winner of the competition will get extra 15% added to their final mark and the other 2 finalists will receive a 5% bonus. Note that your agent may be disqualified from this competition if your agent times out often or for long periods of time. The decision time limit for each agent in the competition will be 1 second per turn.

Written Paper

A report written following one of the report templates (Word or LaTeX) provided in the resources of this module. The report should be **no longer than 4 pages** (two columns). The final report **must be** submitted in PDF format. Please note that these templates provide the format, but you will need to modify sections and section titles to match the ones requested here.

The report must be written in the format and style of an academic paper, in line with the structure of papers that you have been reading during the module. In fact, you can take the Sushi Go! paper (provided in the weekly lab scripts) as a model for the type of content and descriptions that are required on the different sections.

A good guideline for paper structure would be:

- **Abstract (75-125 words):** A single paragraph that outlines what motivates this research, what you did, and a summary of your findings.
- **Introduction (350-450 words):** Initial introduction to the problem, and motivation as to why it is interesting. A brief outline of your approach, and why this is applicable to the problem domain.
- **Background (650-850 words):** The Background section of a paper should explain anything the reader should know as context. This is likely to include:
    - The TAG framework;
    - The game of Sushi Go;
    - The core algorithm(s) you are using, such as MCTS or RHEA. Explain here the algorithm that the submitted agent is based on, including literature review if needed.

  Write this section as if the reader knows nothing of this particular assignment, just some basic knowledge of AI. People outside this class should be able to understand it.

- **Method (700-800 words):** Describe your approach in detail. How does your agent work? If you've included modifications to the original agent, describe them here in detail.

  If you are using MCTS as a base algorithm for example, then you would describe the core algorithm in 'Background', and the changes you made to it (in much more detail) in 'Method'.

  Describe the experimental study performed. What experiments do you run, and why? What information do you hope to get from them?

- **Results (500-700 words):** Present and analyse the results. What do they tell us about the problem. Does the approach work

- **Conclusions and Future Work (200-300 words):** A critical summary of the work and findings of the project, its limitations and potential future work.

- **References (6-15 references):** using an appropriate referencing format (for instance, Harvard, or IEEE). We are not interested in the simple number of references; much more important is their relevance to the approach.

These word counts are only an approximate guide. The two hard constraints are that:

1. you use the correct template (Word or Latex), and;
2. keep within the 4-page limit (references and notes on generative AI use do **not** count against this limit and may spread on to a fifth page).

You also do not have to *religiously* keep to exactly these headings if you think that a slightly different structure would be clearer. For example, some authors like to split a section on 'Previous Work' as a distinct section from Background; or have separate sections for 'Results' and 'Discussion', which have been combined in the outline above. The important point to consider is whether this enhances the clarity of the report and make it easier to follow.

Read the Report Writing Tips uploaded to QMPlus for advice on how to write a report for this assignment and check the AI in Games Sample Report submitted by students from a previous year.

This assignment is excellent practice for your final thesis, which will also need to be in the same format of an academic report, albeit with the luxury of 8 pages.

Individual submission

This consists of a short reflection of the work done during the module. This text should describe in no more than 300 words, the work that each member of the group has done during the project.

The assignment should be submitted as three files to QMplus:

• As a group, a zip file with the source code of the agent (do not include the framework, just the package directory that contains your code changes, such as 'groupA') and a PDF file with the final report. Only one submission per group is necessary.

• The individual reflection on the work done, in a PDF file. One submission per student. Failing to submit your individual reflection will reduce your (individual) mark for this assignment.

4. Marking Criteria

See the separate document for the detailed marking criteria.

You will **not** get extra marks for working on a group of less (or more) than 3 people. (We may form some groups of a different size only if required to fit all students.)

A few points to expand upon from that document:

Complexity of the implemented agent

When thinking about the complexity of your agent, the list below is in rough ascending order of complexity, and hence increased credit. You'll need to have an agent at level 3 or higher on the list below to be able to get 70%+.

1) **Tuning existing agents.**
   You do not need to write any code at all for this assignment and could consider taking one or more existing agents and tuning relevant parameters to be optimal for the specific game environment. This will however preclude you from the highest marks and cap you at about 60%.

2) **Rule-based agents/heuristics.**
   It is possible to write a straightforward rule-based agent for Sushi Go. This is however a course about search and planning algorithms, so this will also preclude you from the highest marks.

3) **Using heuristics/rules within existing algorithms.**
   However, following on from 2), there are many ways that you could use rules/heuristics from your insight in Sushi Go within search/planning algorithms!

4) **Implementation of appropriate algorithmic variants from the literature/lectures.**
   A key word here is appropriate. Just taking a random simple method mentioned in the lectures will not do well. For top marks you need to explain *why* this approach is relevant to Sushi Go in particular, as well as having a good set of experiments and analysis/discussion of their results.

5) **Improvements on algorithms from the literature.**
   The same caveat applies here. If you are enhancing an algorithm beyond the literature, make sure you link the changes to the nature of the game.

Possible topics you might want to consider (this is far from exhaustive, but may give you some ideas):

- *Partial observability*: your agent may tackle the problem of partial observability in games of imperfect information, creating assumptions and managing the fog of war. How does this issue manifest in Sushi Go?

- *Opponent modelling*: your agent might implement an effective (and more complex than Random) system to model the opponent behaviours. This could be either learnt (online or offline) or reactive to the players your agent competes against.
- *Management of the action space*: your agent might try to use pruning to reduce the branching factor of the game, implementing any of the techniques shown in lecture 4.
- *Time management*. If we have 1 second of thinking time per decision, should we behave differently to if we only have 100 milliseconds?
- *Player number*. Does the number of players in a game have an impact? Why?

General clarity

The last part ('General') of the marking criteria refers to the clarity and structure of the report. One thing to avoid is a 'patchwork' report effect, in which each member of the team works on their own idea, runs their own experiments, and then updates the final report separately. This can lead to low marks because of a lack of cohesion and lack of any central structure to the approach. With only four pages this can mean that each individual idea cannot be explained in enough detail.

Better marks are often obtained by teams that try ***fewer*** approaches but therefore have the time to run more exhaustive experiments and more space in the report to do justice to their analysis and discussion.

Tournament

It is not necessary for your approach to do well to get excellent marks. Some of the best reports (and marks) in previous years have been for approaches that failed to do very well in practice, but that were well motivated in theory and with a very good experimental protocol and analysis that helped explain why the agent did poorly. Conversely some people who did well in the competition have been let down by a poor experiment protocol and report. [Having said that, there *is* a positive correlation between doing well in the competition and doing well overall but it is far from deterministic.]

Bonus points from the agent tournament are great but remember that they are exactly that – a bonus. Don't let the competition distract you from the main objective of producing a good report.

Experiments

Whatever your improvement is with respect to the original methods, an appropriate experimental process must be followed in order to show the impact of your modifications. As a general rule, any single experiment should have at least 500 games to get a result that is statistically clear. However, this is a general rule that you are free to break **provided** you justify deviations from it.

The competition between your agents uses 4-player Sushi Go with 500 milliseconds of thinking time per decision. ***You are not constrained to this for your main experiments***. Running 1 experiment with 500 games in each under these conditions might take up to 8 hours; and you will want to run many more than 1 experiment! Factor in your available computing resources. You could for example consider 100ms for initial work and your main set of results and then use

500ms (or more) in a smaller set of experiments. It is required that at least some of your experiments are at the 500ms level; this is not an excuse to run everything with just 10ms; that sort of gaming of the system will be heavily penalised.

Two key things to think about when planning experiments:

Ablation studies

'Ablation' means 'taking stuff away'. Consider the case that you try three improvements to an agent, call them A, B and C. The simplest acceptable experiment would be to compare the performance of this agent against the baseline (and/or other agents that are relevant to the approach).

Better would be to run separate experiments of A vs baseline, B vs baseline and C vs baseline (an ablation of the three conditions).

Even better would be to also run A+B versus the baseline, B+C vs baseline and A+C vs baseline for a full ablation. This can give you lots of useful information on how the three changes interact with each other.

Error analysis

In academic research when you report a result you should always consider the related question of your confidence in the result. The most common approach, and also most relevant to this assignment is to consider what error bars you report.

For example, if an agent wins 240 of 500 games for a 48% win rate, and its opponent wins 260 of 500 for a 52% win rate, how confident are we that the second agent is actually better. A poor answer is to just say the second agent is better. A good answer will provide appropriate confidence bounds on the reported win rate, and comment on what this means. An even better answer will outline the statistical model for the error construction and explain why this is relevant.